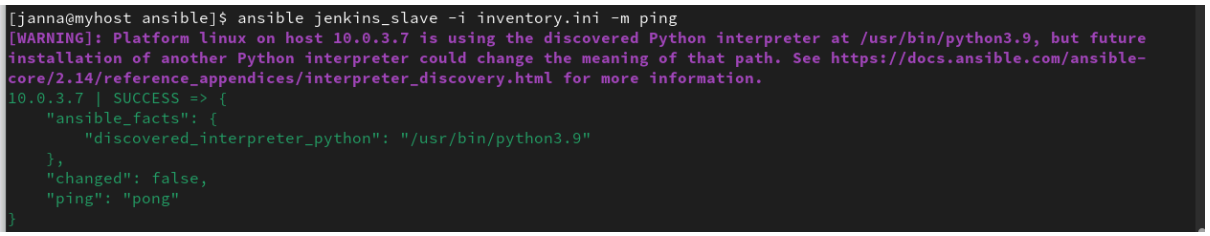
8- configure ansible to run over private ips through bastion (~/.ssh/config)



9- write ansible script to configure ec2 to run as jenkins slave

Configuration as Code with Ansible

With the hop is active, we use Ansible Playbooks to prepare our blank EC2 instances for high-intensity DevOps workloads.

Inventory and Toolchain

Create an Ansible inventory file using the **Private IP** of the Jenkins Slave.

Required Dependencies:

1. **Java 21 (Amazon Corretto):** The specific distribution optimized for Amazon Linux 2023, required for Jenkins agent communication.
2. **Git:** Necessary for the agent to pull the `rds_redis` branch from GitHub.
3. **Docker:** Required to containerize the Node.js application during the build process.

Ansible starts SSH toward `10.0.3.7`

SSH sees the `Host 10.0.*.*` rule in `~/.ssh/config .`

Because of `ProxyJump bastion`, SSH first connects to the bastion

From the bastion, it opens the second SSH connection to `10.0.3.7`

```yaml
tasks:
  - name: Install required tools
    dnf:
      name:
        - java-17-amazon-corretto
        - git
        - docker
      state: present

  - name: Enable and start Docker service
    systemd:
      name: docker
      enabled: true
      state: started

  - name: Add ec2-user to docker group
    user:
      name: ec2-user
      groups: docker
      append: true
```

```
[ec2-user@ip-10-0-3-7 ~]$ git --version
git version 2.50.1
[ec2-user@ip-10-0-3-7 ~]$ java --version
openjdk 17.0.19 2026-04-21 LTS
OpenJDK Runtime Environment Corretto-17.0.19.10.1 (build 17.0.19+10-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.19.10.1 (build 17.0.19+10-LTS, mixed mode, sharing)
[ec2-user@ip-10-0-3-7 ~]$ java --version
openjdk 17.0.19 2026-04-21 LTS
OpenJDK Runtime Environment Corretto-17.0.19.10.1 (build 17.0.19+10-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.19.10.1 (build 17.0.19+10-LTS, mixed mode, sharing)
[ec2-user@ip-10-0-3-7 ~]$ docker --version
Docker version 25.0.14, build 0bab007
```

```
[ec2-user@ip-10-0-3-7 ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[ec2-user@ip-10-0-3-7 ~]$
```

10- configure slave in jenkins dashboard (with private ip)

i used JNLP because my Jenkins server was running locally in a Docker container, while the Jenkins agent was running on a private EC2 instance.

Since the private EC2 instance did not have a public IP, Jenkins could not easily connect to it using SSH.
With JNLP, the connection works in the opposite direction: the agent connects to Jenkins.

Simple steps

1. In Jenkins, I created a new node and selected “Launch agent by connecting it to the controller.”
2. On the private EC2 instance, I created a working directory for the agent.
3. I made sure Java was installed on the EC2 instance because Jenkins agents need Java to run.
4. From the Jenkins node page, I downloaded the agent.jar file.
5. Jenkins generated a JNLP command containing:
    - the Jenkins URL
    - a secret token
    - the agent name
6. I ran that command on the EC2 instance.

After that, the agent connected back to Jenkins and the node appeared as connected.

Why I used JNLP

I used JNLP because the Jenkins server could not directly reach the private EC2 instance.
Instead of Jenkins connecting to the agent, the agent initiated the connection to Jenkins, which made it work even though the instance only had a private IP.

```
[ec2-user@ip-10-0-3-132 node]$ echo c7209828538b6970d1debff0e293f02b105a3484e573ebed426c8160ef86d5c9 > secret-file;curl -sO https://pellet-sway-underrate.ng
k-free.dev/jnlpJars/agent.jar;java -jar agent.jar -url https://pellet-sway-underrate.ngrok-free.dev/ -secret @secret-file -name "private-slave" -webSocket -
rkDir "/home/ec2-user/node"
y 11, 2026 2:12:00 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
FO: Using /home/ec2-user/node/remoting as a remoting work directory
y 11, 2026 2:12:01 PM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
FO: Both error and output logs will be printed to /home/ec2-user/node/remoting
y 11, 2026 2:12:01 PM hudson.remoting.Launcher createEngine
FO: Setting up agent: private-slave
y 11, 2026 2:12:01 PM hudson.remoting.Engine startEngine
FO: Using Remoting version: 3355.v388858a_47b_33
y 11, 2026 2:12:01 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
FO: Using /home/ec2-user/node/remoting as a remoting work directory
y 11, 2026 2:12:04 PM hudson.remoting.Launcher$CuiListener status
FO: WebSocket connection open
y 11, 2026 2:12:04 PM hudson.remoting.Launcher$CuiListener status
FO: Connected
```

```
[ec2-user@ip-10-0-3-132 node]$ cat /etc/systemd/system/jenkins-agent.service
[Unit]
Description=Jenkins Agent
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/node
ExecStart=java -jar /home/ec2-user/node/agent.jar   -url https://pellet-sway-underrate.ngrok-free.dev/   -secret c7209828538b6970d1debff0e293f02b105a3484e573ebed426c8160ef86d5c9   -name "pri
vate-slave"   -webSocket   -workDir "/home/ec2-user/node"
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
[ec2-user@ip-10-0-3-132 node]$
```

11- create pipeline to deploy nodejs_example fro branch (rds_redis)

I added jenkinsfile in the nodejs_app

```
pipeline {
    // This tells Jenkins to run this job specifically on your private EC2 slave
    agent {
        label 'private-ec2'
    }

    // This section securely pulls the changing Terraform outputs from Jenkins Credentials
    environment {
        RDS_HOSTNAME = credentials('rds-url-secret')
        RDS_USERNAME = credentials('rds-user-secret')
        RDS_PASSWORD = credentials('rds-pass-secret')
        REDIS_HOSTNAME = credentials('redis-url-secret')
    }

    stages {
        stage('Pull Code') {
            steps {

                checkout scm
            }
        }


        stage('Build') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub', usernameVariable: 'USERNAME', passwordVariable: 'PASSWORD')]) {
                sh '''
                docker build . -t jannawael/nodejs_terr:latest
                docker login -u ${USERNAME} --password ${PASSWORD}
                docker push jannawael/nodejs_terr:latest

                '''

                }
            }
        }

        stage('deployment ') {


              steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub', usernameVariable: 'USERNAME', passwordVariable: 'PASSWORD')]) {
                sh '''
                docker login -u ${USERNAME} --password ${PASSWORD}
                docker run -d -p 80:3000 -e RDS_HOSTNAME="$RDS_HOSTNAME" -e RDS_USERNAME="$RDS_USERNAME" -e RDS_PASSWORD="$RDS_PASSWORD" -e RDS_PORT="3306" -e REDIS_HOSTNAME="$REDIS_HOSTNAME" -e REDIS_PORT="6379" jannawael/
nodejs_terr:latest

                '''

            }
            }
        }
    }
```

```
Masking supported pattern matches of $PASSWORD
[Pipeline] {
[Pipeline] sh
+ docker login -u jannawael --password ****
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
+ docker run -d -p 3000:3000 -e RDS_HOSTNAME=**** -e RDS_USERNAME=**** -e RDS_PASSWORD=**** -e RDS_PORT=3306 -e REDIS_HOSTNAME=**** -e REDIS_PORT=6379
jannawael/nodejs_terr:latest
3b3e617f6e6ba668607c8dff69e37d93879bc4e20e279e845d6f00dc269945a2
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
[ec2-user@ip-10-0-3-132 node]$ docker ps
CONTAINER ID   IMAGE                           COMMAND                  CREATED         STATUS         PORTS
          NAMES
3b3e617f6e6b   jannawael/nodejs_terr:latest   "docker-entrypoint.s…"   9 seconds ago   Up 7 seconds   0.0.0.0:3000->3000/tcp, :::3000->3
000/tcp   agitated_jennings
[ec2-user@ip-10-0-3-132 node]$
```

12- add application load balancer to your terraform code to expose your nodejs app on port 80 on the load balancer

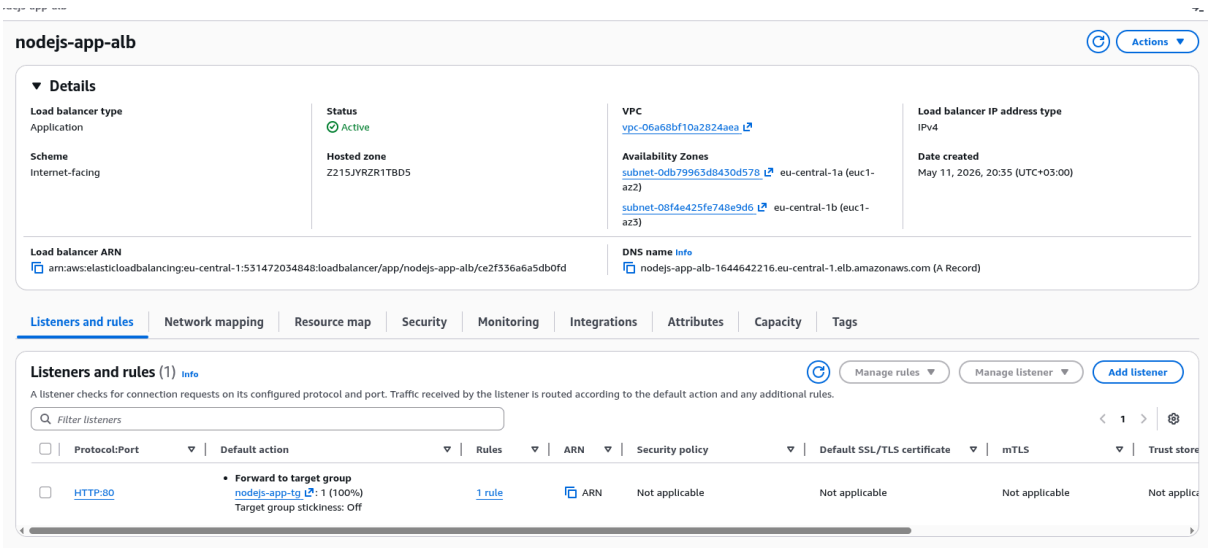
. **Create the ALB Security Group** You must create a new security group specifically for the load balancer. It needs an **Inbound Rule** that allows HTTP traffic on Port 80 from anywhere (`0.0.0.0/0`) so users on the internet can reach it.

2. Update your Private EC2 Security Group To maintain maximum security, you should update the security group of your private EC2 instances (your Jenkins slaves). They should allow inbound traffic on Port 80, but **only** from the ALB's security group. This guarantees that no one can bypass the load balancer to talk to your servers directly.

3. Create the Target Group Define an `aws_lb_target_group` resource that listens on Port 80. Then, use an `aws_lb_target_group_attachment` to point this target group to the **Private IPs** of the EC2 instances where your Jenkins pipeline deployed the Node.js Docker container. *(Note: Because your `docker run` command mapped `-p 80:3000`, the target group must send traffic to Port 80 on the instance).*

4. Create the Application Load Balancer Define the `aws_lb` resource. It is critical that you place the ALB in your **Public Subnets** so it can receive internet traffic. Attach the ALB security group you created in step 1 to this load balancer.

5. Create the ALB Listener Add an `aws_lb_listener` resource to your ALB. Configure it to listen on Port 80 and set its default action to **forward** the incoming traffic to the Target Group you created in step 3.



```
aws_lb.nodejs_alb: Still creating... [01m50s elapsed]
aws_lb.nodejs_alb: Still creating... [02m00s elapsed]
aws_lb.nodejs_alb: Still creating... [02m10s elapsed]
aws_lb.nodejs_alb: Still creating... [02m20s elapsed]
aws_lb.nodejs_alb: Creation complete after 2m23s [id=arn:aws:elasticloadbalancing:eu-centra
l-1:531472034848:loadbalancer/app/nodejs-app-alb/ce2f336a6a5db0fd]
aws_lb_listener.http_listener: Creating...
aws_lb_listener.http_listener: Creation complete after 1s [id=arn:aws:elasticloadbalancing:
eu-central-1:531472034848:listener/app/nodejs-app-alb/ce2f336a6a5db0fd/e12cd72249c35ffc]

Apply complete! Resources: 5 added, 1 changed, 0 destroyed.

Outputs:

alb_dns_name = "nodejs-app-alb-1644642216.eu-central-1.elb.amazonaws.com"
[janna@myhost terrlab1]$
```

12- test your application by calling loadbalancer_url/db and /redis

db connection successful

redis is successfuly connected